

Vulkax Physics Studio: A Reproducible Equation-to-Visualization Foundation

Vulkax Research Project

July 2026

Abstract

Vulkax Physics Studio is a native desktop foundation for converting symbolic scalar equations and deterministic simulation graphs into inspectable visual fields, reproducible exports, and future GPU compute workloads. The project preserves the prior BEACON, GeoBEACON, and Atlas renderer work as regression research systems, but reorients the active product toward physics visualization. This report documents implemented evidence: a canonical expression AST, a CPU reference evaluator, a scalar GLSL compute-contract emitter, deterministic wave and reaction-diffusion references, a Schwarzschild null-ray integrator with a thin-disk visualization, a buoyant-smoke stable-fluid reference, a Qt 6 editor with persistent Vulkan Wave Field and Gray-Scott execution, and reproducible PNG sequence export. It separates these verified components from unimplemented claims such as persistent GPU ownership for every simulation graph, HDR accumulation, and film-grade general-relativistic rendering.

1 Motivation

Large map and navigation systems require data services, legal data pipelines, routing, search, network operations, and global content delivery. Those are worthwhile engineering problems, but they obscure the research question of a visual-computing project. Physics Studio instead has a direct pipeline: an equation or simulation graph is the input, an interactive scientific/cinematic field is the output, and measured numerical, visual, and time budgets determine quality.

The imagery of *Interstellar* motivates a standard of physical honesty, not a parity claim. The film's DNGR renderer used specialized ray-bundle techniques and a production offline workflow. Vulkax therefore labels each result as analytical reference, CPU simulation reference, future GPU compute output, or offline export rather than presenting all visuals as equivalent.

2 System

```
Equation text / presets
|
v
Canonical AST ----> CPU reference evaluator ----> Qt field preview / PNG export
|
+-----> GLSL scalar-field compute contract ----> persistent editor Vulkan Wave Field
|
SimulationGraph CPU references (wave, Gray-Scott) ----> persistent Gray-Scott display plus live MSE
```

The scalar parser supports constants, variables, arithmetic, right-associative powers, signed powers, and a bounded function set including sine, exponent, square root, clamp, min, and max. Expression validation reports a source column on parse failure. The GLSL compiler traverses the same AST, checks every free variable against the field coordinate and parameter bindings, then emits a storage-buffer compute shader contract.

The first hardware compute gate dispatches a fixed wave-field storage-buffer shader and reads it back for comparison. On the Apple M2 Pro at 64×36 samples, the measured MSE was 1.90944×10^{-13} and the maximum absolute error was 8.69864×10^{-7} against the CPU reference. This validates that specific shader path; it does not establish every AST-generated expression or dynamic simulation graph as GPU-equivalent.

The same wave result was then regenerated from the canonical AST through the equation GLSL emitter, compiled by `glslangValidator`, and dispatched through the same Vulkan runner. Its MSE and maximum error were unchanged. This establishes a checked equation-to-SPIR-V-to-GPU path for the wave preset, while leaving coverage of every other preset as a separate validation requirement.

The Qt 6 Quick application is a native macOS bundle. Its primary workflow contains a preset library, equation editor, parameter controls, timeline, field viewport, native project open/save dialogs, and PNG or numbered PNG sequence export. The application asks Qt Quick for the Vulkan RHI backend on interactive launches. Wave Field and Gray-Scott also own a separate persistent compute-only Vulkan context, retaining its device, pipelines, buffers, command buffer, fence, and timestamp query for the editor session before readback into the Qt image provider. Gray-Scott rebuilds from the deterministic seed on a parameter edit or seek, displays GPU secondary concentration, and compares it to a CPU solver reference. This is real GPU execution, but it is not yet shared-resource Qt RHI integration or GPU ownership for every graph.

3 Reference Simulations

The wave reference uses a second-order explicit update with zero-valued boundaries. The reaction-diffusion reference uses a bounded Gray-Scott update with two ping-pong fields. Both implementations expose their update contracts as generated compute-kernel source stubs, allowing a future Vulkan implementation to compare each cell against a deterministic CPU result.

The wave graph has a checked Vulkan ping-pong implementation. A 64×64 field executed for 32 steps matched the CPU field cell-for-cell in the recorded Apple M2 Pro run (MSE and maximum absolute error both reported as zero). The two-field Gray-Scott implementation was then run on the same hardware for 64 steps. It measured 7.04×10^{-16} MSE for field *A*, 5.51×10^{-18} MSE for field *B*, and 1.19×10^{-7} maximum absolute error. Both are headless storage-buffer/readback comparisons. The Gray-Scott run additionally recorded 6.73471 ms of summed Vulkan compute timestamp time, excluding CPU submission and readback. These values do not imply that the current Qt viewport is already driven by persistent GPU simulation resources.

4 Particle Gravity Reference

The editor includes a deterministic two-dimensional softened Newtonian N-body reference. It uses kick-drift-kick velocity Verlet integration and initializes orbiters in the center-of-mass frame, so total linear momentum begins at zero. A 4,800-step regression bounds relative energy drift below two percent and repeats the same particle positions bit-for-bit for the same seed. The native editor renders the resulting bodies and exports a checked 24-frame PNG sequence with frame variation validation. This is intentionally a CPU reference and raster preview; it is not yet a GPU particle simulation or instanced rendering claim.

That reference now has a matching two-pass Vulkan compute implementation. Each time step writes a kick-drift intermediate buffer, establishes a compute read/write barrier, and evaluates the second kick into the next buffer. A seven-body Apple M2 Pro run for 128 steps measured 1.86×10^{-12} MSE and 5.45×10^{-6} maximum component error against the CPU state, with 15.1534 ms summed compute timestamp time. This is a headless correctness and timing result; the editor has not yet adopted persistent GPU particle buffers or instanced drawing.

The relativity module integrates the equatorial Schwarzschild null-ray equation $u'' + u = 3Mu^2$ using Runge-Kutta four in geometric units ($G = c = 1$). Rays whose impact parameter is at or below $\sqrt{27}M$ are reported as captured at the photon sphere. This is an intentionally narrow reference module: it does not claim Kerr spacetime, ray bundles, accretion-disk rendering, or a complete black-hole film renderer.

The editor's Schwarzschild lensing preset builds a quadratic-spaced RK4 deflection lookup, maps escaped rays to an inclined emissive annulus, and applies a documented Doppler-brightness heuristic. The capture boundary remains driven by the photon-sphere criterion. Its source-plane mapping and brightness model are visual approximations, deliberately separated from the ray equations, so the preview is not represented as a Kerr, ray-bundle, or full physical black-hole image.

The buoyant-smoke example is a deterministic two-dimensional stable-fluid solver. Each fixed time step advects velocity, density, and temperature semi-Lagrangianly; applies buoyancy and vorticity confinement; and projects velocity with Jacobi pressure iterations. Its regression checks deterministic replay, finite values, bounded divergence, source mass, and upward plume transport. This is an incompressible 2D simulation and preview, not a volumetric combustion or 3D smoke claim.

5 Measured Artifacts

The first analytical run evaluated six shipped presets for 180 frames with 512 samples per frame. The reported energy proxy is the mean squared scalar value, not a physical energy calculation.

Table 1: Checked CPU analytical reference run.

Preset	Min	Max	Energy proxy
Wave field	-1.0000	1.0000	0.499596
Gravity potential	-14.3088	-1.9808	36.049642
Quantum wavepacket	-0.8751	1.0000	0.132575
Electromagnetic pulse	-1.0000	1.0000	0.233030
Reaction-diffusion seed	0.0000	0.9999	0.133218
N-body orbit potential	-19.6151	-0.2500	6.166781

The sequence exporter generated 24 frames at 30 fps and 640×360 pixels with a self-contained manifest that records the expression, preset, timing, dimensions, and filenames. Figure 1 shows the first field frame.



Figure 1: Wave-field frame from the checked Physics Studio PNG sequence.

For the Schwarzschild reference, the weak-field ray at $b = 100M$ produced a deflection of 0.04122 radians, close to the leading-order $4M/b = 0.04$ radians. At $b = 5M$, below $\sqrt{27}M$, the module reports capture rather than a fabricated outgoing path. Near the critical impact parameter the deflection rises sharply, as expected for the simplified equatorial model.

6 Verification

The checked CTest suite covers AST precedence and error handling, AST-to-GLSL emission, deterministic simulation updates, Schwarzschild weak-field and capture behavior, controller hysteresis, controller and QML application smoke paths, sequence export, legacy BEACON behavior, city-data reproducibility, and the preserved Atlas application bundle. The raw artifacts are stored under `docs/results/physics_studio_current/`.

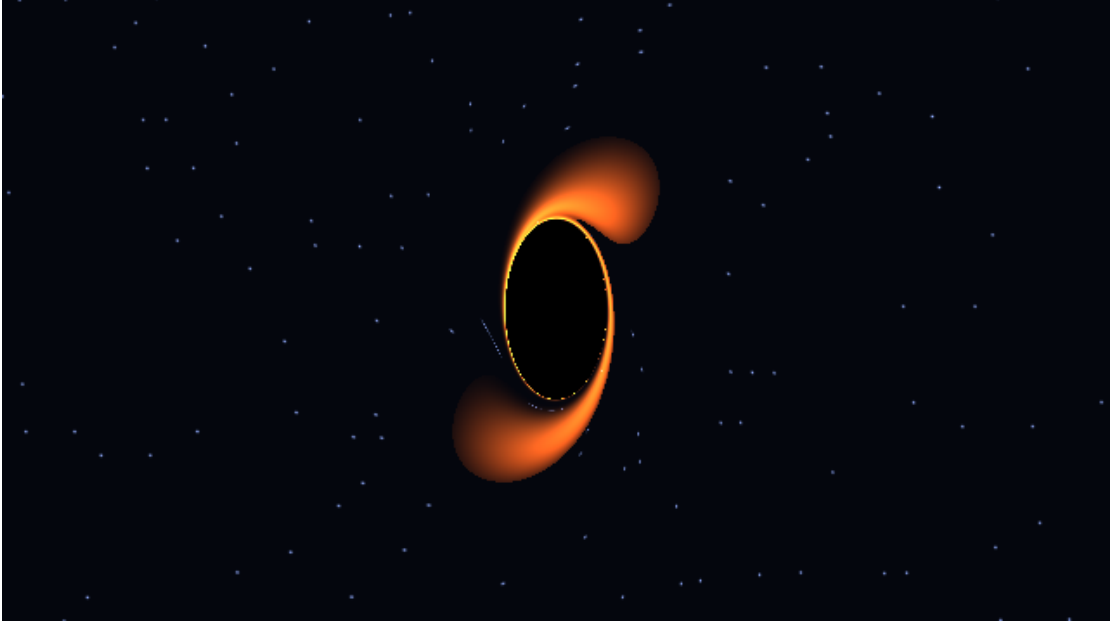


Figure 2: Checked Schwarzschild thin-disk frame. Captured rays form the central shadow; disk placement derives from an RK4 deflection lookup, with a heuristic brightness model.

7 Quality-Controller Experiment

The quality controller now has a reproducible fixed-versus-adaptive analytical preview benchmark. It evaluates the canonical quantum-wavepacket AST at a full reference grid and at the policy’s selected grid, then records per-frame sampling time, nearest-grid MSE, resolution, samples, and EWMA state. On the checked 120-frame Apple M2 Pro run with a 1 ms target, fixed preview produced a 9.55 ms p50 with zero resampling MSE. The adaptive policy produced a 1.19 ms p50, 0.00162 mean MSE, and 13 changes. This demonstrates the controller’s measurable timing-quality trade-off for a CPU analytical workload; it is not a GPU, perceptual, or rendered-image result. The same fixed-grid sampling MSE now feeds the native editor’s controller for analytical presets. The dynamic Gray–Scott path instead reports live GPU/CPU solver MSE. Particle and lensing paths deliberately leave that image-error signal unmeasured rather than substituting an unrelated equation reference.

Dynamic graph parameters are part of the project contract. Reaction–diffusion exposes diffusion, feed, and kill; the N-body graph exposes central mass, orbiter mass, and softening. Any such change rebuilds deterministic state at the current timeline time. A dedicated project smoke test stores a modified reaction configuration at 0.5 seconds, changes it, and verifies that reopening restores the intended parameters and time.

8 Scope Boundaries

The current repository proves CPU/GPU agreement and Vulkan timestamp measurements only for the checked wave, Gray–Scott, and seven-body particle workloads. It does *not* yet establish general numerical convergence against closed-form solutions, persistent editor GPU ownership beyond Wave Field and Gray–Scott, HDR accumulation, camera/director tools, full 3D volumes, Kerr geodesics, or an adaptive controller connected to live renderer timings. The native bundle does export a linearized half-float OpenEXR preview, but it is not represented as a full HDR radiance or offline-rendering pipeline. Those are necessary future gates, not implicit achievements. The complete phase and acceptance criteria are maintained in `docs/VULKAX_PHYSICS_STUDIO_ROADMAP.md`.

9 Conclusion

Vulkax has a runnable native desktop editor and a reproducible, testable physics-visualization foundation. Its immediate value is not a claim of a finished scientific renderer; it is the unification of equation semantics, reference computation, export provenance, and future GPU contracts in one inspectable codebase. This establishes a credible

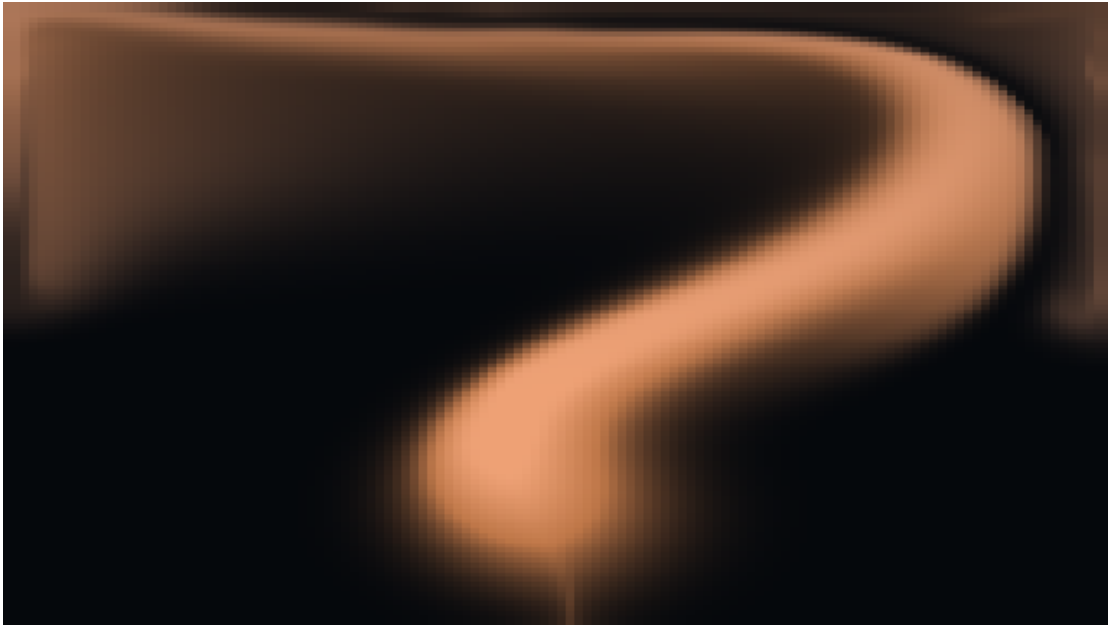


Figure 3: Checked buoyant-smoke frame from the deterministic two-dimensional incompressible solver.

base for the next research gate: persistent GPU simulation ownership for the dynamic graphs with timestamped quality/time trade-offs.